西南交通大學
Southwest Jiaotong University

云计算技术

第7讲 分布式存储系统

课程代码：SCAI012112

授课教师：戴朋林

2026年4月



1 分布式存储基础理论

- CAP
- BASE
- 一致性模型

03 CAP定理

西南交通大学 Southwest Jiaotong University

CAP 定理（Brewer's Theorem）：由 UC Berkeley 的 Eric Brewer 于 2000 年提出，2002 年由 Gilbert & Lynch 正式证明。

核心结论：分布式系统中，一致性（C）、可用性（A）、分区容错（P）三者最多同时满足两个。

💡 类比：餐厅同时做到"菜品新鲜（C）、上菜飞快（A）、食材断货也不停业（P）"——现实中三者很难兼顾，总要取舍。

特性	英文	含义
一致性 (C)	Consistency	所有节点同一时刻看到相同数据。 例：银行转账后，所有查询立即看到新余额
可用性 (A)	Availability	每个请求都收到响应（不保证是最新数据）。 例：系统总能回答，即使答案可能稍旧
分区容错 (P)	Partition Tolerance	节点间通信故障时系统仍可运行。 例：机房网线断了，各节点独立继续服务

04 CAP定理

西南交通大学 Southwest Jiaotong University

🤔 **为什么只能三选二？** 网络故障（P）在真实环境中无法避免。分区发生时，节点 A 和节点 B 断联——此时只有两条路：

✓ 选 C（一致性）：拒绝响应，等节点联通后再服务 → 牺牲可用性（A❌）

✓ 选 A（可用性）：继续服务，可能返回旧数据 → 牺牲一致性（C❌）

选择	含义	典型系统
CP	网络分区时拒绝服务，确保数据绝对一致 ——宁可不响应，不返回脏数据	ZooKeeper（分布式协调） etcd（K8s 配置中心）、HBase
AP	网络分区时继续响应，允许短暂数据不一致 ——电商库存展示可短暂偏差	Cassandra、DynamoDB Couchbase（购物车、用户动态）
CA	理论上不存在：真实分布式系统无法完全避免网络分区，CA 仅适用于单机场景	MySQL / PostgreSQL（单节点部署） 注：集群模式下也要面对 P

05 CAP定理的现实理解

西南交通大学 Southwest Jiaotong University

误解	纠正
CAP 三选二	不是静态"三选二"。P（网络分区）几乎不可避免， 实际是在分区发生时选 C 还是选 A
AP 系统完全不一致	AP 系统提供的是最终一致性，不是永远不一致。 例：朋友圈点赞数会在几秒内同步到所有人
CP 系统完全不可用	CP 系统只在分区期间拒绝服务，分区恢复后立刻恢复。 例：ZooKeeper 短暂不可用后自动重新选主恢复

📖 **延伸：PACELC 模型**（Daniel Abadi 提出）：CAP 只关注"出故障时怎么选"。系统正常运行时同样面临**延迟（L）vs 一致性（C）**的权衡——是等所有副本同步后再响应（一致但慢），还是立即返回（快但可能读到旧数据）？

06 CAP定理的现实理解

西南交通大学 Southwest Jiaotong University

PACELC 读法： 发生网络分区（P）时 → 选可用（A）还是一致（C）？ | 正常运行（E=Else）时 → 选低延迟（L）还是强一致（C）？

系统	分区时 P → A or C?	正常时 E → L or C?
DynamoDB	**PA**（优先可用）	**EL**（低延迟优先）
Cassandra	**PA**（优先可用）	**EL**（低延迟优先）
HBase	**PC**（优先一致）	**EC**（一致性优先）
MongoDB	**PA**（优先可用）	**EC**（一致性优先）

07 BASE理论

西南交通大学 Southwest Jiaotong University

BASE 是对 CAP 中 AP 路线的进一步阐述，是大型互联网系统（淘宝、微信、亚马逊）常用的设计理念。 **核心思想：放弃强一致性，换取高可用和高性能。**

缩写	全称	含义
BA	Basically Available	允许部分功能降级或响应延迟，但系统整体依然可用。 例：淘宝大促时搜索变慢、但下单功能正常
S	Soft State	数据不需要时刻一致，允许副本间短暂不同步。 例：微博点赞数在各数据中心可短暂不一致
E	Eventually Consistent	停止写入后，经过一段时间所有副本数据趋于一致。 例：DNS 更新后，全球各地 IP 解析逐步同步到新值

08 BASE理论

西南交通大学 Southwest Jiaotong University

ACID vs BASE ——两种截然不同的数据库设计哲学

维度	ACID	BASE
全称	原子性 / 一致性 / 隔离性 / 持久性	基本可用 / 软状态 / 最终一致
一致性	强一致性	最终一致性
可用性	可能降低（等待同步）	高可用
适用场景	银行转账、订单支付	社交动态、商品库存展示
代表系统	MySQL、PostgreSQL	DynamoDB、Cassandra

⚠️ BASE 并非"放任数据随便乱"，而是采用**最终一致性**，需要设计**冲突解决机制**（如读修复、反熵协议、向量时钟）。

💡 选择依据：需要强一致性（金融/订单）→ 选 ACID；需要高可用高吞吐（社交/推荐）→ 选 BASE。

09 一致性模型

西南交通大学 Southwest Jiaotong University

一致性级别（从强到弱）——强度越高，延迟越大

模型	定义	延迟	典型系统
线性一致性	读操作必须返回最新写入值，所有操作按实时顺序排列。例：etcd 写配置后全集群立即可见	最高（全副本同步）	ZooKeeper、etcd
顺序一致性	所有操作按同一全局顺序执行，不保证实时性。例：Raft 日志所有节点顺序一致	高（共识协议）	Raft/Paxos 共识
因果一致性	有因果关系的操作保序，无关操作可乱序。例：先发帖后评论，顺序可见	中（追踪依赖）	MongoDB(Causal)
最终一致性	停止写入后，最终所有副本收敛到同一值。例：DNS 更新、微博点赞数	低（异步传播）	DynamoDB、Cassandra
读己写一致性	用户一定能读到自己刚写入的数据。例：发帖后刷新自己主页能看到新帖	低（会话级）	常见优化策略

10 一致性模型

西南交通大学 Southwest Jiaotong University

线性一致性（Linearizability）——最强的一致性

系统行为就像只有一个副本：写入完成后，所有后续读操作必须返回新值。

代价：每次写入需等所有副本确认，延迟高。典型系统：etcd、ZooKeeper

时间轴示例（线性一致性）：

```
Client A: Write(x=1) ———— 完成
Client B:              Read(x) → 必须返回 1（不能返回旧值 0）
```

最终一致性（Eventual Consistency）——最弱的一致性

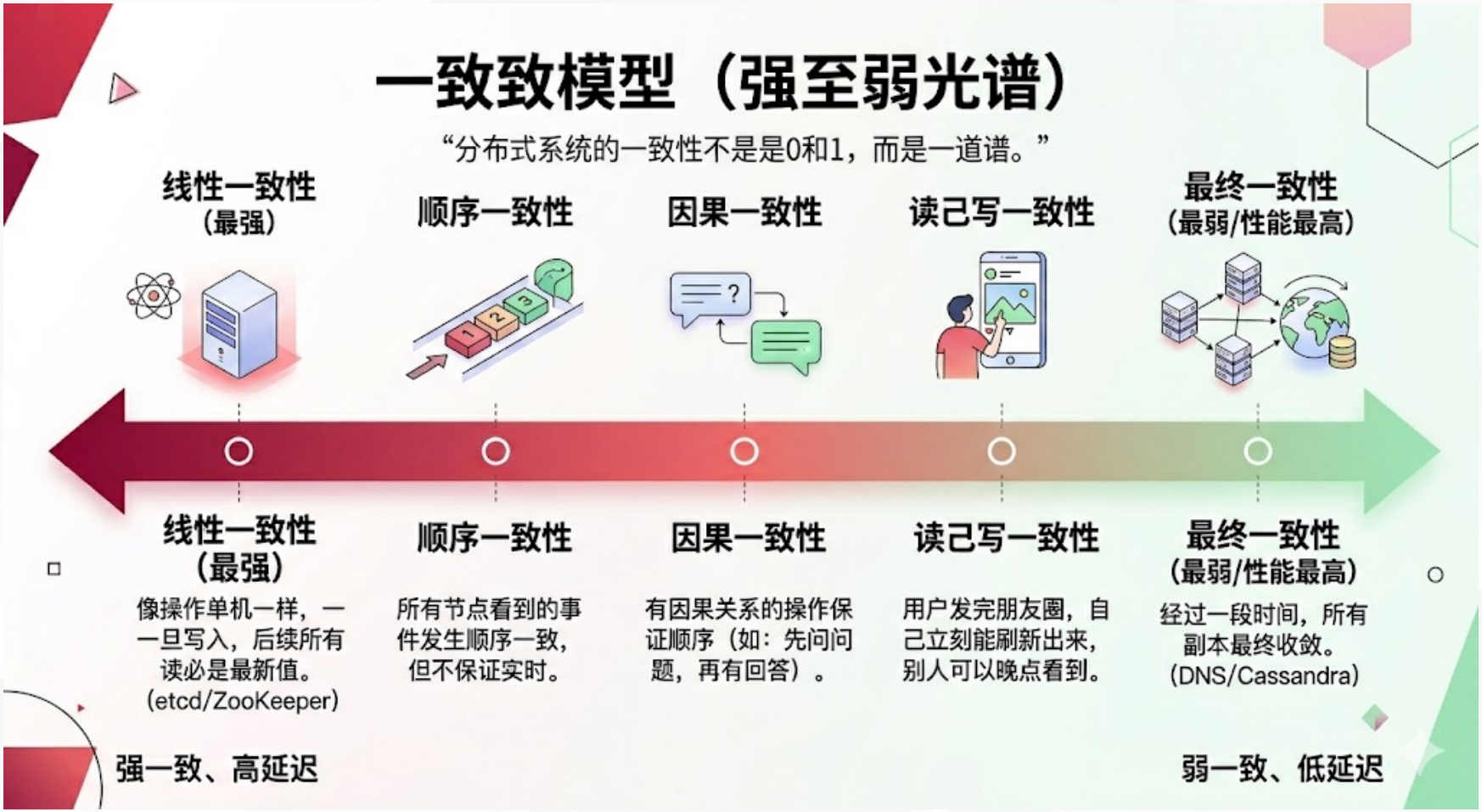
允许短暂不一致，但停止写入后所有副本最终会收敛到相同值。

优势：低延迟、高可用。典型系统：DynamoDB、Cassandra、DNS 解析

时间轴示例（最终一致性）：

```
Write(x=1) → 副本 A: x=1（立即写入）
             副本 B: x=0（暂未同步）   副本 C: x=0（暂未同步）
一段时间后 → 副本 A/B/C 全部：x=1（最终一致 ✓）
```

一致性模型


一致致模型（强至弱光谱）
“分布式系统的一致性不是是0和1，而是一道谱。”
线性一致性
(最强)
顺序一致性
因果一致性
读己写一致性
最终一致性
(最弱/性能最高)
线性一致性
(最强)
像操作单机一样，一一旦写入，后续所有读必是最新值。
(etcd/ZooKeeper)
顺序一致性
所有节点看到的事件发生顺序一致，但不保证实时。
因果一致性
有因果关系的操作保证顺序（如：先问问题，再有回答）。
读己写一致性
用户发完朋友圈，自己立刻能刷新出来，别人可以晚点看到。
最终一致性
(最弱/性能最高)
经过一段时间，所有副本最终收敛。
(DNS/Cassandra)
强一致、高延迟
弱一致、低延迟

2 数据分片策略

- 数据分片
- 哈希分片
- 虚拟节点

13 为什么需要数据分片?

西南交通大学 Southwest Jiaotong University

数据分片（Sharding）： 将大规模数据集分割成多个子集并分布存储在不同节点，每个节点只负责部分数据的读写，实现水平扩展、突破单机瓶颈。

💾 容量瓶颈	⚡ 性能瓶颈	⚠️ 可用性风险
存储上限：单机磁盘有限，无法应对 PB 级数据	读写 QPS 存在物理上限，单机无法撑住高并发	单点故障风险高，一台宕机即全库不可用
📌 微信图片存储 > 100 PB，单机根本装不下	📌 淘宝双11峰值写入 > 1亿次/秒，远超单机极限	📌 单库 MySQL 故障 → 整个业务停摆

■ 两大分片策略

🔢 哈希分片（Hash Sharding）

按 Key 的哈希值对节点数取模，决定数据存入哪个节点

✅ 优点：分布均匀，负载均衡好

⚠️ 缺点：节点数变化时大量数据需迁移（扩容代价高）

📌 代表：Redis Cluster（CRC16 % 16384）

📏 范围分片（Range Sharding）

按 Key 的值范围划分区间，同一范围的 Key 存入同一节点

✅ 优点：范围查询高效（顺序扫描）

⚠️ 缺点：容易产生数据倾斜（热点节点）

📌 代表：HBase、BigTable、Google Spanner

14 哈希分片（Hash Sharding）

西南交通大学 Southwest Jiaotong University

■ 核心原理

对数据的 Key 计算哈希值，然后对节点数 N 取模，决定数据存储在哪个节点：

node_id = hash(key) % N

例：hash("user:1001") % 3 = 2 → 存入 Node 2

```
// 哈希分片核心公式
node_id = hash(key) % N
// 3节点示例
hash("user:1001") % 3 = 2  // → Node 2
hash("order:5523") % 3 = 0  // → Node 0
// Redis Cluster 实际使用
slot = CRC16(key) % 16384
```

✅ 主要优点

- 数据均匀分布各节点，有效实现负载均衡
- 写入压力分散，避免单节点热点
- 代表：Redis Cluster 用 CRC16(key) % 16384 分配 16384 个槽位

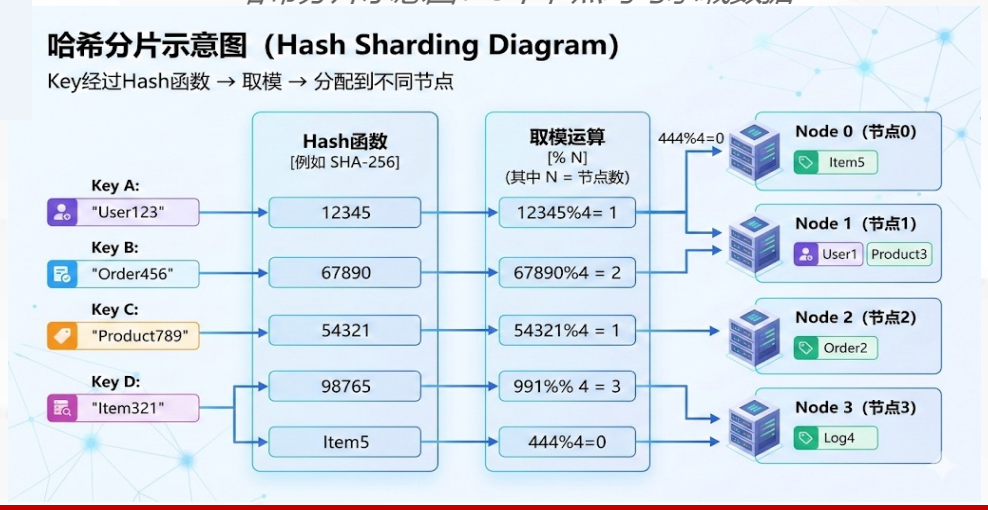
⚠ 致命缺点

节点数 N 变化时，几乎所有数据映射关系都改变，引发大规模数据迁移：

- 3台 → 4台：hash(key)%3 变为 hash(key)%4
  约 75% 的数据需要重新迁移！
- 无法支持高效范围查询（需扫描所有节点）

💡 解决方案：使用一致性哈希（→ 下一页）

▲ 哈希分片示意图：3个节点均匀承载数据



15 一致性哈希（Consistent Hashing）

西南交通大学 Southwest Jiaotong University

■ 设计目标

解决传统哈希节点增减时大量数据迁移的问题：

- 3台→4台，传统哈希迁移约 75% 数据
- 一致性哈希只需迁移约 1/N ≈ 25% 数据

■ 核心原理（Hash Ring）

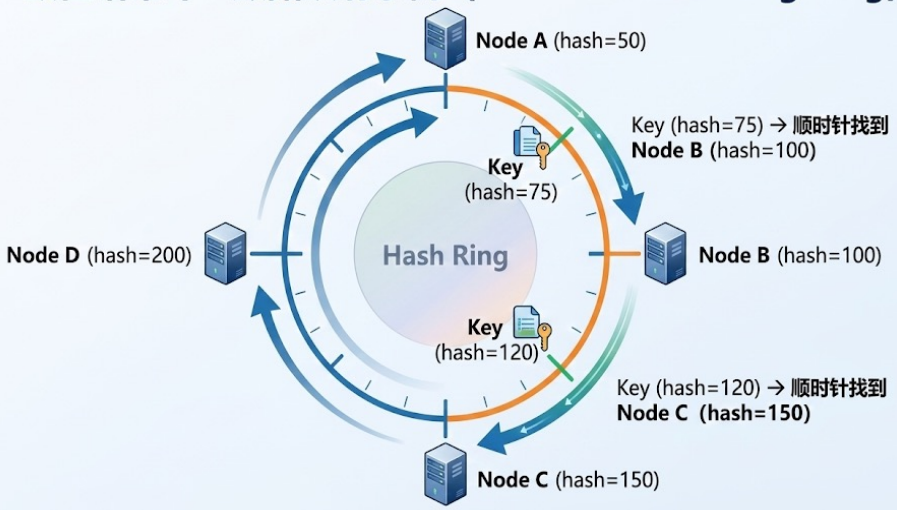
哈希空间组成虚拟圆环（0 ~ $2^{32}-1$），节点和 Key 均映射到环上。

- Key 顺时针找最近节点存入
- 类比：钟表盘——数据找顺时针方向最近的刻度（节点）

✅ 节点增减优势

删节点：只将该节点数据迁给顺时针下一节点
加节点：只将新节点覆盖区间的数据迁入
结论：扩/缩容只影响 1/N 的数据！
虚拟节点：每个物理节点映射多个虚拟节点，使分布更均匀
代表系统：Cassandra、DynamoDB、Riak

一致性哈希环：数据映射示例（Consistent Hashing Ring）



▲ *哈希环示例*

Node A/B/C 均匀分布在环上，数据 key 顺时针找最近节点存入。

典型系统： Cassandra、DynamoDB、Riak

16 一致性哈希 (Consistent Hashing)

西南交通大学 Southwest Jiaotong University

对比	传统哈希	一致性哈希
增删节点	几乎所有数据迁移 (N-1)/N ≈ 91%（10→11节点）	只影响相邻节点 约 1/N ≈ 9%（10→11节点）
迁移量公式	~(N-1)/N × 总数据量 10个节点扩到11个：迁移91%	~1/N × 总数据量 10个节点扩到11个：仅迁移9%
查询路由	直接计算：node = hash(key) % N 节点数变化则公式失效	顺时针找最近节点 节点增减不影响其他区段路由

💡 **类比：** 快递公司新增一个配送站，只需把新站附近的包裹移交，其他线路完全不受影响。
传统哈希就像所有包裹都要重新分配站点——加一个站点，全员搬家，代价极高。

📊 **数字说话（10 → 11 个节点）：** 传统哈希迁移 ≈ 90% 数据 **一致性哈希仅迁移 ≈ 9%** ✓

17 虚拟节点（Virtual Nodes）

西南交通大学 Southwest Jiaotong University

⚠ **一致性哈希的遗留问题**：节点在哈希环上的落点是随机的，少量节点时分布容易不均，某个节点承接了一大片区间，成为热点。

解法：虚拟节点（VNode）

每个物理节点在哈希环上注册多个"分身"，这些分身均匀散落在环的各处。数据照样顺时针找最近的节点，只不过现在找到的是某个分身，实际由对应的物理节点处理。

钟表盘比喻：3台服务器，各放3个贴纸，9个贴纸均匀贴满表盘——每台服务器大约各管三分之一。

```
节点 A  →  A-1(hash=10)   A-2(hash=80)   A-3(hash=190)
节点 B  →  B-1(hash=40)   B-2(hash=120)  B-3(hash=230)
节点 C  →  C-1(hash=60)   C-2(hash=150)  C-3(hash=250)
```

虚拟节点越多，分布越均匀。实践中通常每个物理节点对应 **100～200 个虚拟节点**，Cassandra、DynamoDB 均采用此方案。

18 范围分片（Range Sharding）

西南交通大学 Southwest Jiaotong University

原理：按 Key 的值范围将数据划分到不同节点，每个节点负责一段连续区间。查找数据时直接定位到对应区间的节点，无需遍历。

示例（按用户名首字母分片）：

```
Node A: key ∈ [A, G)  →  alice, ben, fan
Node B: key ∈ [G, N)  →  han, kim, max
Node C: key ∈ [N, Z]  →  nao, sam, zhang
```

热点问题与解法：

若 Z 开头有大量明星账号，Node C 成热点 → 可对热点区间再细分，或引入二级哈希打散。

典型系统：HBase（RowKey 范围自动分区）、Spanner、BigTable

✅ 优点

- 范围查询高效——只访问 1 个节点（如查所有 A-F 用户）
- 数据有序存储，天然支持排序和分页
- 分区边界可灵活调整（自动或手动再分裂）

⚠️ 缺点

- 数据倾斜——热点 Key 集中时某节点压力过大
- 需要维护分区元数据（映射表），扩容要重新分裂区间

📌 适用场景

- 时序数据（日志、监控，按时间段查）
- 用户数据按 ID 段分片（用户量均匀分布时）
- 需要频繁做范围扫描的 OLAP 场景

19 分片策略对比总结

西南交通大学 Southwest Jiaotong University

🔵 哈希分片（Hash Sharding）

- 公式：node = hash(key) % N
- ✅ 数据分布均匀，避免热点
- ⚠️ 不支持范围查询；扩容需大量迁移
- 改进：一致性哈希 + 虚拟节点（迁移量降至 1/N）
- 典型：Redis Cluster（16384槽）、DynamoDB、Cassandra

🟣 范围分片（Range Sharding）

- 公式：按 Key 值区间分配节点
- ✅ 范围查询高效；有序存储；支持排序分页
- ⚠️ 数据倾斜时产生热点；需维护分区元数据
- 典型：HBase（RowKey 自动分区）、BigTable、Spanner

🎯 如何选择？

选哈希分片，当：

- 需要均匀分散写压力（如用户行为日志）
- 主要是点查（按 Key 精确查找），不做区间扫描
- 例：用户账户系统、缓存集群、消息队列

选范围分片，当：

- 业务依赖范围查询（如按时间段查日志）
- 数据有天然顺序、需要排序或分页
- 例：时序数据库、订单按日期分区、监控指标

💡 实践中两者常结合使用：先按业务维度哈希分片，再在单片内按时间范围分区（二级分片）。

3 数据复制策略

- 数据复制模式
- Quorum机制
- 冲突解决策略

21 数据复制模式

西南交通大学 Southwest Jiaotong University

为什么需要复制?

高可用：节点宕机时副本顶上，服务不中断
低延迟：将副本部署在离用户近的地域
读扩展：多副本分担读请求，大促时吞吐翻倍

三种复制模式

① **主从**（Leader-Follower）：主库写，从库只读。MySQL / PostgreSQL
② **多主**（Multi-Leader）：多地域写入，需解决写冲突。CouchDB
③ **无主**（Leaderless）：任意节点可写，靠 Quorum 保一致。DynamoDB

① 主从复制详解

写操作只发给主库，主库把变更日志（binlog）同步给所有从库。从库延迟通常在毫秒级。

✅ 优点：实现简单，读扩展容易，主流 RDB 默认支持
⚠️ 缺点：主库宕机需选举新主，写入期间短暂不可用

② 多主复制详解

多个数据中心各有主库，本地写入后跨中心同步。适合多地域部署场景。

⚠️ 难点：两地同时修改同一条记录 → 写冲突，需设计解决策略

③ 无主复制详解

写请求同时发给多个节点，达到 Quorum（W 个节点确认）即算成功。无单点瓶颈。

✅ 高可用，节点随时可加入/离开。DynamoDB、Cassandra 的核心机制

22 三种复制模式

西南交通大学 Southwest Jiaotong University

1. 主从复制（Leader-Follower）

```
写请求 → Leader → 同步复制 → Follower 1    ← 等待 Follower 确认后才返回，强一致
                  → 异步复制 → Follower 2    ← 先返回再慢慢同步，低延迟但可能丢数据
读请求 → Leader 或 Follower（由客户端策略决定）
```

类别	具体内容
特点说明	写操作只能发给 Leader，确保单点写入顺序一致；读操作可以路由给 Follower，实现读写分离
优点	实现简单，容易理解；Follower 可分担读流量，水平扩展读吞吐量；Leader 挂掉后可从 Follower 中选举新主
缺点	Leader 是写入单点，写吞吐量受限；异步复制时 Follower 可能有延迟，读到旧数据（复制滞后）
代表系统	MySQL / PostgreSQL / Redis / MongoDB 副本集 / Kafka（每个 Partition 有 Leader）

23 多主复制（Multi-Leader Replication）

西南交通大学 Southwest Jiaotong University

允许多个节点同时作为 Leader 接收写入，各 Leader 之间异步互相同步并向全部 Follower 共享数据。

■ 工作原理

- 客户端就近写入最近的 Leader，降低跨地域延迟
- 写入后异步同步至其他 Leader 和 Follower
- 并发写入同一条记录会引发 **写冲突**，需内置合并策略

■ 典型应用场景

- **多数据中心**：各地区各启一个 Leader，用户就近访问
- **离线应用**：手机日历/笔记，网络恢复后同步至云端
- **常见技术**：CouchDB、MySQL GR、Google Docs

■ 优势

✓ 写入延迟低：就近 Leader 写入，无需跨地域跌杠

✓ 容灯高：任意 Leader 故障其他仍可继续写入

✓ 水平扩展：多 Leader 分担写压力、提升写内吞吐量

■ 挑战与代价

⚠ **写冲突**：两 Leader 同时改同数据，需内置关定合并策略

⚠ **复制延迟**：异步属于最终一致，各 Leader 短期可能不同步

⚠ **实现复杂**：需额外写冲突检测和合并逻辑

24 无主复制（Leaderless Replication）

西南交通大学 Southwest Jiaotong University

没有主节点，客户端同时向多个副本写入和读取，通过 Quorum 投票机制保证数据一致性。

■ 工作原理

- 写入：客户端同时将数据写入 n 个副本中的 w 个，收到 w 个确认即成功
- 读取：同时向 r 个副本发起读请求，取最新版本号的值返回
- **Quorum 条件：** w + r > n，保证读集和写集必有交叉

■ 典型代表与优势

- **代表技术：** Cassandra、DynamoDB、Riak
- 无单点故障：任意节点故障不影响写入
- 适合高可用、大规模分布式写入场景

■ Quorum 数学解析

- n = 副本数，w = 写联答数，r = 读联答数
- **w + r > n**：读集与写集必有交叉，一定能读到最新写
- 常用配置 n=3, w=2, r=2：允许 1 个节点故障

■ 代价与局限

- ⚠ **读延迟高：** 同时读 r 个节点，耗时取最慢响应
- ⚠ **写冲突：** 并发写入没有主规合并，需最后写入胜等策略
- ⚠ **最终一致：** 不保证强一致性，仅实现最终一致

25 Quorum 机制

西南交通大学 Southwest Jiaotong University

Quorum（法定人数）：读写操作必须得到足够多节点确认才算成功，保证数据一致性。

N = 副本数　　**W** = 写联答数　　**R** = 读联答数　　**强一致性条件：W + R > N**

■ **常见配置示例（N = 3）**

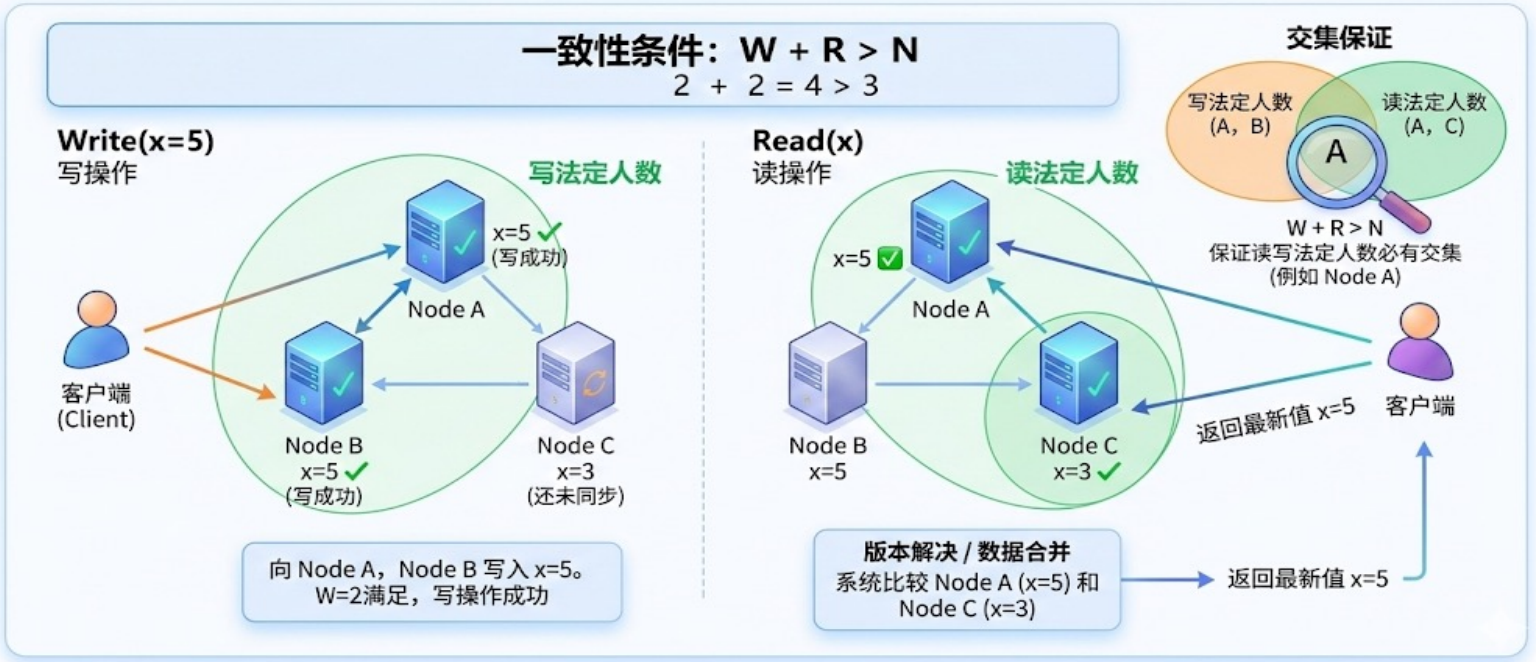
配置	W	R	特点
强一致读写	2	2	W+R=4>3，任何读都能读到最新写入
写优先	3	1	写入所有副本，读取一个即可
读优先	1	3	写入一个即返回，读取所有副本
最终一致	1	1	高性能，但可能读到旧数据

26 Quorum 机制

西南交通大学 Southwest Jiaotong University

Quorum机制示意图

N=3（总副本数），W=2（写入法定人数），R=2（读取法定人数）



- 进阶：Sloppy Quorum

27 冲突解决策略

西南交通大学 Southwest Jiaotong University

当多个客户端同时向同一 Key 写入不同值时，产生写冲突。以下是常见的四种解决方案：

■ **常见冲突解决方案对比**

方案	原理	优点	缺点
最后写入胜出（LWW）	取时间戳最大的写入	简单	可能丢失并发写入
向量时钟	记录每个节点的版本号，检测并发冲突	精确检测冲突	实现复杂
CRDTs	数据结构自动合并，无冲突	无需冲突解决	只适用于特定数据类型
应用层解决	返回所有冲突版本，由应用合并	灵活	增加应用复杂度

28 向量时钟示例

向量时钟（Vector Clock）记录每个节点的写入次数，用于检测并发写入和冲突。下为具体演示过程：

① 初始状态

x 的向量时钟初始化为零

版本：{A:0, B:0}

→

② Node A 写入 x=1

A 节点写入，A 的计数 +1

版本：{A:1, B:0}

③ Node B 并发写入 x=2

B 节点同时写入，B 的计数 +1

版本：{A:0, B:1}

→

④ 检测到并发冲突

{A:1,B:0} 与 {A:0,B:1} 互不包含

⚠ **冲突！需应用层合并 → {A:1, B:1}**

结论：向量时钟能精确检测并发写入，但冲突合并需交由应用层处理

29 CRDT

西南交通大学 Southwest Jiaotong University

Conflict-free Replicated Data Type：并发修改多副本无冲突、自动收敛到一致状态的特殊数据结构。

■ 典型 CRDT 类型

G-Counter
只增计数器：每节点各自递增，合并取最大值，用于统计点击数、浏览量等

PN-Counter
可增可减计数器：用两个 G-Counter 分别记录加减，用于库存数量

G-Set
只添加集合：取各副本并集，适合标签系统、白名单等只增场景

OR-Set
可添删集合：用唯一标记区分添加与删除操作，协同文档中的元素列表

■ 核心特性

- **无冲突设计：**数据结构内置合并逻辑，并发写入永远不冲突
- **最终一致：**所有副本在网络恢复后必然收敛到相同状态
- **可交换并可结合：**操作顺序不影响最终结果

■ 适用场景与局限

- 实时协作编辑（Google Docs 光标位置、文字内容）
- 购物车、点赞数、在线状态等高并发计数场景
- ⚠ **局限：**只适用特定数据结构，通用性不如向量时钟

4 分布式键值存储

- Redis Cluster
- Amazon DynamoDB
- etcd

31 Redis Cluster

西南交通大学 Southwest Jiaotong University

Redis 分布式方案，通过哈希槽（Hash Slot）将数据自动分布到多节点，支持自动分片和高可用。

特性	说明
分片方式	哈希槽（Hash Slot），共 16384 个槽
槽分配	CRC16 (key) % 16384 确定 Key 属于哪个槽
高可用	每个主节点有从节点，主故障时自动切换
去中心化	无中心节点，节点间通过 Gossip 协议通信

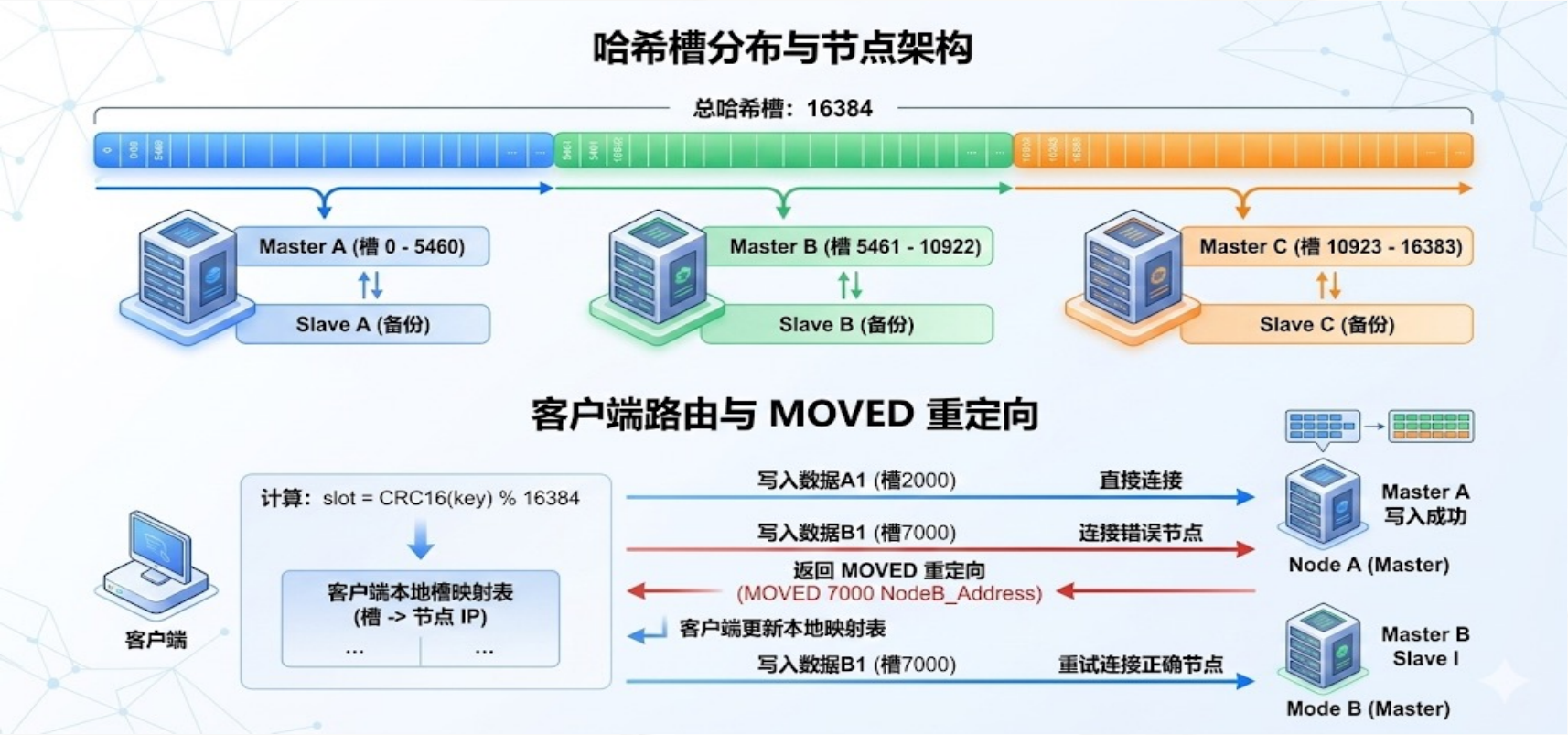
■ 核心架构特点

- **16384 个哈希槽：** 每个主节点负责一部分槽位，平均分散读写压力
- **主从复制：** 每个主节点配备备份，主故障时自动选主，无需手动干预
- **Gossip 心跃：** 节点间相互传播健康状态，无中心节点，天然去中心化

■ 优势与注意事项

- ✓ 水平扩展简单，动态加节点无需停机
- ✓ 适合大规模高并发缓存，可支持 1000+ 节点
- ⚠ **注意：** 多键事务和跨槽 Lua 脚本受限

32 Redis Cluster架构图



■ **局限性：不支持跨槽多 Key 事务**（可用 Hash Tag 将相关 Key 分配到同一槽）、**不支持跨槽 Pipeline**、扩缩容时需手动迁移槽位。

33 Amazon DynamoDB

西南交通大学 Southwest Jiaotong University

AWS 提供的全托管、无服务器的 NoSQL 数据库，设计说明来自 2007 年 Amazon Dynamo 论文。

■ 一、是什么？DynamoDB 的定义与数据模型

数据按 **Table** 组织，每条记录称 **Item**（包含若干 **Attribute** 键值对），模式自由；必须指定 **Partition Key**，可选添加 **Sort Key** 实现范围查询。

■ 二、底层架构：分片、复制与一致性

- **Partition Key** 哈希后决定分片，每片在 3 个 AZ 内同步复制，少数 AZ 确认后返回写成功。
- **一致性可选：** 默认最终一致读（延迟低），可切换强一致读（在主片读，延迟略高）。

■ 三、三大核心能力

全托管	个位数毫秒延迟	全球多活
无需管理服务器、分片和复制，AWS 自动运维	读写延迟稳定在毫秒级，支持每秒百万级请求	Global Tables 跨 Region 多活，自动双向复制

34 DynamoDB：数据模型与一致性读取

西南交通大学 Southwest Jiaotong University

一致性选项

```
Table: Orders                          // 订单表，模式自由
  Partition Key: order_id              // 必选，哈希决定分片
  Sort Key: timestamp                  // 可选，支持范围查询
  Attributes: { user_id, amount, status, ... }  // 字段灵活，各 Item 可不同
Primary Key = Partition Key [+ Sort Key]  // 唯一标识一条记录
```

■ 二、一致性选项对比

选项	说明	延迟
最终一致读	可能读到稍旧的数据（默认）	**更低**
强一致读	保证读到最新写入	**稍高**
事务	支持 ACID 事务（跨表）	**最高**

35 DynamoDB vs Redis：技术选型对比

西南交通大学 Southwest Jiaotong University

维度	DynamoDB	Redis
存储	磁盘（SSD）	内存
持久性	强（三副本）	需配置 AOF/RDB
数据模型	Key-Value + Document	丰富数据结构(String/Hash/List/Set/ZSet)
运维	全托管	需自运维或使用托管服务
成本	按请求 / 容量计费	按内存计费

■ 选型建议

✓ 选 DynamoDB 的场景

- 需持久化、大规模存储（TB 级）
- 全托管、无需运维，降低 DBA 成本
- 需要 ACID 事务或跨表查询

⚡ 选 Redis 的场景

- 需极低延迟（微秒级）缓存 / 会话存储
- 使用 ZSet 排行榜、List 消息队列等丰富结构
- 数据量小、对内存成本不敏感

36 etcd

西南交通大学 Southwest Jiaotong University

由 **CoreOS** 开发，是 **Kubernetes** 默认的集群数据存储组件

■ 什么是 etcd

- **本质：**一个分布式的键值存储，类似一个高可靠的"共享字典"
- **定位：**Kubernetes 集群的"唯一数据源"，保存所有集群状态和配置
- **保障：**基于 Raft 共识算法，确保多个节点间的数据始终一致

■ 核心能力

- **Watch 机制：**监听数据变化，变更时自动推送通知给订阅者
- **MVCC：**多版本并发控制，支持查询任意历史版本的数据
- **租约（Lease）：**Key 可设过期时间，用于分布式锁和服务注册

37 etcd

西南交通大学 Southwest Jiaotong University

Raft 共识算法——让多台机器对同一份数据达成一致

■ 基本思想

- **核心问题：**多台机器如何确保“看到同一份数据”，不出分歧?
- **解决思路：**选一位 Leader 统一处理写入，其他节点只负责跟随同步
- **容错能力：**过半节点存活即可正常工作(如 3 台挂 1 仍能运行)

■ 工作流程

- **① Leader 选举：**收不到心跳 → 发起投票 → 多数票者当选 Leader
- **② 日志复制：**写请求交给 Leader → 同步给多数 Follower → 确认提交
- **③ 心跳维持：**Leader 定时广播心跳，Follower 收到即保持跟随

38 etcd

西南交通大学 Southwest Jiaotong University

etcd 在 Kubernetes 中的角色 —— 集群的“唯一数据源”

■ 数据流向

- **唯一入口：** 所有组件通过 API Server 读写 etcd，禁止直接访问
- **调用链路：** kubectl / Controller / Scheduler → API Server → etcd
- **设计优势：** 统一入口便于鉴权、审计和并发控制

■ etcd 中存储的关键数据

- **资源对象：** Pod / Service / Deployment / ConfigMap 等完整状态
- **集群配置：** 网络策略、存储卷、调度规则等
- **服务发现：** Endpoint 地址与服务注册信息
- **权限数据：** RBAC 角色与权限绑定策略

5 列族存储

- Google BigTable模型
- HBase
- Apache Cassandra

40 Google BigTable模型

Google BigTable是Google于2006年发表论文描述的分布式存储系统，用于管理PB级别的结构化数据。它是HBase和Cassandra等系统的设计原型。

BigTable是一个稀疏的、分布式的、持久化的多维有序Map。

(Row Key, Column Family:Column Qualifier, Timestamp) → Value

概念	说明	示例
Row Key	行键，按字典序排列	com.google.www
Column Family	列族，一组相关列的集合	content、anchor
Column Qualifier	列限定符，列族内的具体列	content:html、anchor:cnn
Timestamp	时间戳，每个 Cell 可存多个版本	t1=1234567890
Cell	由 (RowKey, Column, Timestamp) 唯一确定的值	网页 HTML 内容

41 Google BigTable模型

西南交通大学 Southwest Jiaotong University

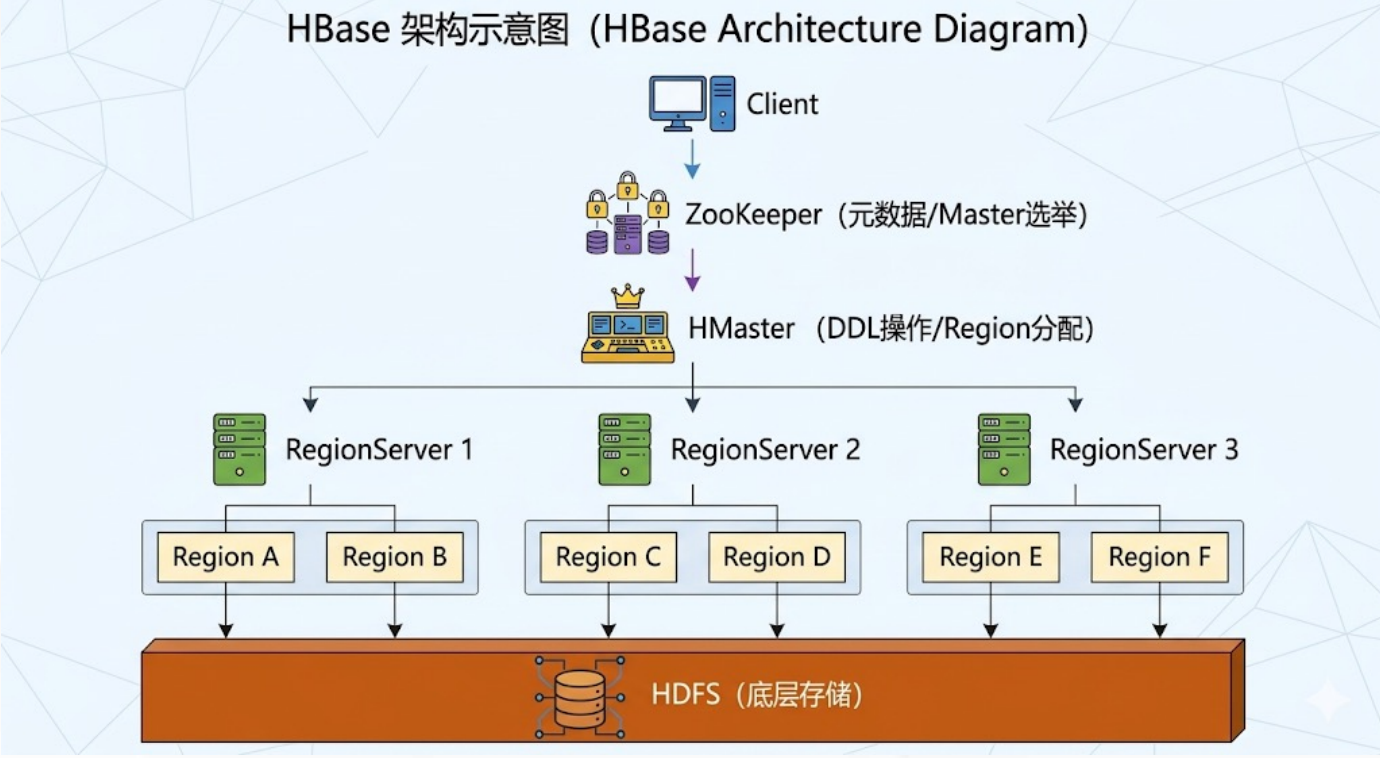
示例：网页存储

```
Row Key: "com.google.www"
  Column Family "content":
    content:html   → {t3: "<html>...", t2: "<html>...", t1: "<html>..."}
  Column Family "anchor":
    anchor:cnn     → {t1: "Google Homepage"}
    anchor:bbc     → {t1: "Google Search"}
```

- 行键采用反向域名 com.google.www，使同域名的网页在表中聚集存放，方便范围扫描
- 两个列族：content 存放网页正文，anchor 存放指向本页的外链（cnn、bbc 等），结构清晰
- 同一单元格保留多版本（t1/t2/t3），可回溯网页历史快照，无需覆盖旧数据

42 HBase

Apache HBase是Google BigTable的开源实现，基于HDFS 实现强一致、高可靠的海量列族存储，是Hadoop 生态中最主流的随机读写 NoSQL 数据库。



43 HBase 核心组件

西南交通大学 Southwest Jiaotong University

概念	说明
Region	表按 Row Key 范围分割成 Region，分布在不同 RegionServer
RegionServer	负责 Region 的读写，类似数据节点
HMaster	管理 Region 分配、DDL 操作
MemStore	内存写缓冲（每个 Region 一个）
HFile	磁盘存储格式（SSTable），不可变
WAL	Write-Ahead Log，防止 MemStore 数据丢失

44 HBase

西南交通大学 Southwest Jiaotong University

写入流程（先日志、后内存、再刷盘）

```
Write → WAL（持久化日志）
      → MemStore（内存）
      → 当MemStore满 → Flush为HFile（磁盘）
      → HFile积累 → Compaction（合并）
```

适用场景

- 海量数据：单表可达百亿行 × 百万列，支持 PB 级列族存储
- 时序与日志：监控指标、访问日志、消息流等需高吸入、按时间查询的场景
- 随机读写 + 范围扫描：按 Row Key 范围快速扫描，低延迟人点访问

45 Apache Cassandra

西南交通大学 Southwest Jiaotong University

Apache Cassandra是由 Facebook 开发并捐赠给 Apache 的开源分布式 NoSQL 数据库，融合了 BigTable 的列族数据模型与 Amazon Dynamo 的去中心化架构，天生多数据中心部署，以 “无单点故障 + 线性扩展” 着称。

核心特性

特点	说明
去中心化	无 Master 节点，所有节点对等（Peer-to-Peer）
高可用	AP 系统，写入永不拒绝
线性扩展	增加节点即增加性能
可调一致性	每次读写可选择一致性级别
分布式	数据自动分片和复制

Apache Cassandra

示例：可调一致性（Tunable Consistency）

```
# Cassandra可调一致性
# 写入时指定
INSERT INTO orders (id, amount) VALUES (1, 99.0)
  USING CONSISTENCY QUORUM;  -- 写多数节点

# 读取时指定
SELECT * FROM orders WHERE id = 1
  CONSISTENCY LOCAL_ONE;     -- 只读本地一个节点
```

- **QUORUM**：写入只有多数节点确认才算成功，越严格 → 越一致但延迟越高。
- **LOCAL_ONE**：只读本地一个副本，违完强退化 → 速度极快但可能读到旧数据。
- “可调”：一致性由开发者按请求选择，在强一致、高可用与低延迟之间灵活权衡。

47 Cassandra vs HBase 对比

西南交通大学 Southwest Jiaotong University

维度	Cassandra	HBase
CAP 选择	AP（高可用）	CP（强一致）
架构	去中心化（P2P）	中心化（Master-Slave）
写性能	极高（无 Master 瓶颈）	高
读性能	高	高（Region 级缓存）
运维	简单（无 Master）	复杂（ZK+HMaster+HDFS）
查询语言	CQL（类 SQL）	Java API / HBase Shell
依赖	独立运行	依赖 HDFS + ZooKeeper

一句话选型：写多读多、强一致、就有大数据栏和 Hadoop 生态 → **HBase**；要高可用、多数据中心、桌面式简单运维 → **Cassandra**。

48 列族存储对比总览

西南交通大学 Southwest Jiaotong University

维度	HBase	Cassandra	Google Bigtable（云服务）
开发者	Apache	Apache (Facebook)	Google
数据模型	BigTable	BigTable + Dynamo	BigTable
一致性	强一致（CP）	可调一致性（AP）	强一致
架构	Master-Slave	P2P 去中心化	全托管
扩展性	好	极好（线性）	极好
运维	复杂	中等	无需运维
生态	Hadoop 生态	独立	GCP
适用场景	大数据分析 + 随机读写	高写入 + 全球分布	GCP 大数据

选型口诀：已有 Hadoop 生态、要强一致 → **HBase**；全球部署、要高可用 → **Cassandra**；用 GCP、要免运维 → **Cloud Bigtable**；需结构化查询则考虑 NewSQL。

6 分布式数据库（NewSQL）

- NewSQL
- Google Spanner
- TiDB

NewSQL的定义与背景



■ 是什么？NewSQL 的定义

- 兼备 SQL 的关系型模型与 ACID 事务
- 同时具备 NoSQL 的水平扩展能力
- “鱼与熊掌兼得”的新一代关系型数据库

■ 为什么出现？背景与动机

- 传统 SQL：扩展性差，难以支撑海量写入
- NoSQL：放弃事务，应用层值平高
- 云原生腰力指数级富需求新层次

■ 代表产品

Google Spanner
全球分布式 SQL，无限扩展，TrueTime 保证外部一致性

CockroachDB
开源 NewSQL，兼容 PostgreSQL，跨 Region 多活

TiDB
国产 NewSQL，MySQL 兼容，HTAP 混合负载

51 NewSQL 核心特性解析

西南交通大学 Southwest Jiaotong University

NewSQL 是新型分布式数据库，兼备传统 SQL 的 **ACID 事务** 与标准 SQL 接口，同时具备 NoSQL 的**水平扩展**能力与高可用。

特性	技术原理	代表实现
SQL 兼容	标准 SQL 语法，支持 JOIN / 子查询 / 事务 DDL	TiDB、CockroachDB、Spanner
ACID 事务	分布式 2PC + Percolator / Raft 日志保证原子性	Spanner(TrueTime)、TiDB
水平扩展	Range/Hash 分片，自动 Rebalance，线性写扩展	TiKV、CockroachDB Ranges
高可用	多副本 Raft/Paxos，自动 Leader 选举与故障切换	TiDB PD、CockroachDB
强一致性	全局时间戳（TrueTime / HLC）保证线性一致读	Spanner、TiDB TSO

52 Google Spanner：全球分布式数据库

西南交通大学 Southwest Jiaotong University

Google Spanner 首个实现全球范围**线性一致性 + 分布式 ACID** 事务的系统，2012 年 OSDI 论文深刻影响了数据库领域。

TrueTime 机制 TrueTime 基于 GPS + 原子钟的全球精确时间 API，返回区间不确定性 **ε ≈ 1–7 ms**，Commit 时等待 ε 确保全局偏序。

```
TrueTime.now() → [earliest, latest]     // 返回时间区间，非精确点
时间不确定性 ε ≈ 1–7 ms（通常 <1 ms）     // GPS + 原子钟保障
Spanner Commit 时等待 ε ，确保事务时间戳全局偏序     // commit-wait 机制
```

■ 三大核心优势

全球线性一致性

跨数据中心事务具备外部一致性，读取结果反映全局真实顺序

- 若 T1 在 T2 开始前提交，则任何节点读到的 T2 结果必然包含 T1 的修改
- 依赖 TrueTime 原子钟 + GPS 保证全球时间戳单调递增

自动分片 & 负载均衡

基于范围分片，Directory 自动迁移 Tablet，动态均衡写入压力

- 数据分布横跨多 Zone、多 Region，无需手动 resharding
- 支持每个表独立配置副本数与地理分布策略（Placement）

真正的 ACID 事务

TrueTime + 2PC + Paxos 保证分布式 ACID，无需业务端妥协

- Paxos 组内多数派确认写入，单节点宕机不影响事务提交
- 支持跨表、跨行、跨 Region 的全局串行化（Serializable）隔离

53 Google Spanner：系统架构层级

西南交通大学 Southwest Jiaotong University

组件	说明
Spanserver	数据节点，管理 Tablet（数据分片）
Zone	部署单元，类似 BigTable 的服务器集群
Universe	Spanner 全局部署，包含多个 Zone
Paxos Group	每个分片使用 Paxos 协议复制，保证强一致

■ 架构层级（从外到内）

Universe
Spanner 全局部署单元
含多个 Zone

Zone
数据中心部署单元
类似 BigTable 集群

Spanserver
数据节点
管理 Tablet 分片

Paxos Group
Paxos 强一致复制
每分片独立副本组

54 Google Spanner：四大核心特性

西南交通大学 Southwest Jiaotong University

全球分布

- 数据跨多个大洲自动分片存储
- 读写操作优先在本地 Zone 完成
- 提供 99.999% SLA（五个九可用性）

外部一致性（External Consistency）

- 比线性一致性更强的全局保证
- TrueTime API 控制时钟误差 ≤7ms
- 事务按提交时间戳全局有序

完整 SQL 支持

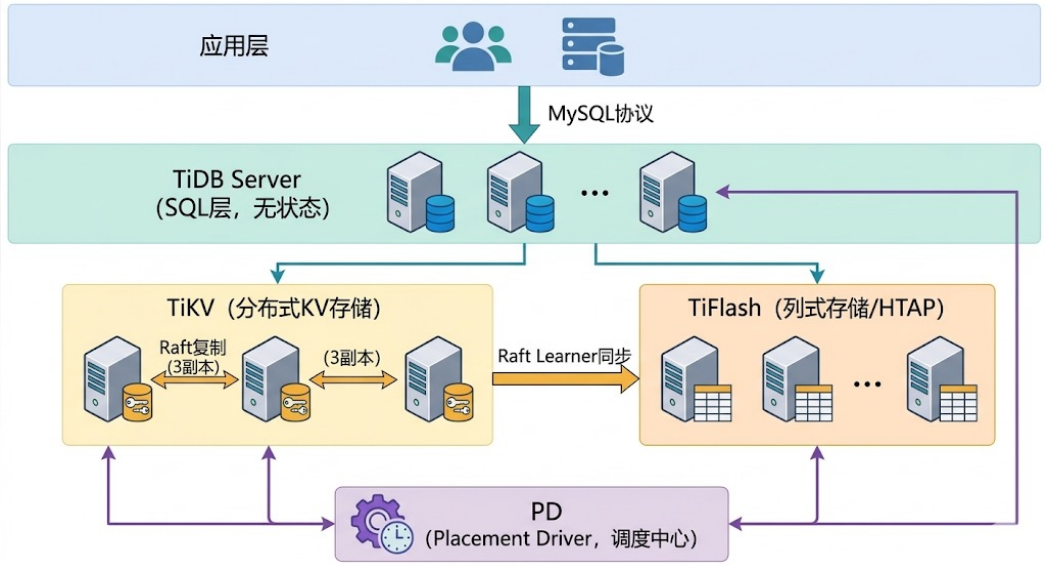
- 支持标准 SQL、JOIN、子查询
- ANSI 2011 SQL 标准兼容
- 支持 Schema 变更（在线 DDL）

云原生全托管

- Cloud Spanner（GCP）全托管服务
- 自动扩缩容，无需手动分片
- 按节点数/处理单元计费

55 TiDB：国产开源 NewSQL 数据库

TiDB 是 **PingCAP** 开发的开源分布式 NewSQL 数据库，兼容 MySQL 协议，设计灵感来自 Google Spanner 和 F1。



■ TiDB 四大核心特点

MySQL 兼容 完全兼容 MySQL 5.7 协议，应用无需修改即可迁移

水平扩展 TiKV 存储层自动分片，线性扩展写入和存储容量

HTAP 支持 TiFlash 列存引擎同时支持 OLTP 写入和 OLAP 分析

高可用 Multi-Raft 协议，自动故障转移，RPO=0 / RTO<30s

TiDB 四大核心组件详解

组件	职责	技术栈
TiDB	SQL 解析、优化、执行（无状态，可水平扩展）	Go 语言编写，Parser+Planner+Executor，协议层兼容 MySQL 5.7
TiKV	分布式 KV 存储引擎（RocksDB + Raft）	Rust 语言编写，底层 RocksDB + Multi-Raft，Region 默认 96MB
PD	全局调度器（时间戳分配、Region 调度、负载均衡）	Go 语言编写，基于 etcd，全局唯一递增时间戳（TSO）
TiFlash	列式存储引擎（实时分析，HTAP）	C++ 编写，ClickHouse 列存格式，与 TiKV 异步复制

■ **TiDB 架构层次关系（客户端请求从上至下流转）**

SQL 计算层	分布式存储	调度与元数据	列式分析层
TiDB Server	**TiKV**	**PD Server**	**TiFlash**
无状态 水平扩展 MySQL 兼容	Raft 副本 96MB Region RocksDB	TSO 时间戳 Region 调度 负载均衡	列存格式 异步复制 OLAP 加速

57 TiDB：MySQL 兼容与分布式事务实战

西南交通大学 Southwest Jiaotong University

■ 左：建表语句（MySQL 兼容）　　右：分布式事务（跨表原子提交）

```
-- MySQL 兼容建表，应用无需改动即可迁移
mysql -h tidb-server -P 4000 -u root
CREATE TABLE orders (
  id BIGINT PRIMARY KEY AUTO_INCREMENT, //
分布式自增
  user_id INT,
  amount DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
);
```

```
-- 分布式事务：两表原子提交（2PC）
START TRANSACTION;

INSERT INTO orders (user_id, amount)
  VALUES (1, 99.00);  // 写入订单

UPDATE inventory SET stock = stock - 1
  WHERE sku = 'A001';  // 扣减库存

COMMIT;  // 全部成功或全部回滚
```

NewSQL 三强对比：Spanner / TiDB / CockroachDB

维度	Google Spanner	TiDB	CockroachDB
开发者	Google	PingCAP	Cockroach Labs
兼容性	Google SQL	MySQL	PostgreSQL
共识算法	Paxos	Raft	Raft
全球分布	原生支持（TrueTime）	支持	支持
HTAP	Cloud Spanner 支持	TiFlash 列式引擎	有限支持
部署	仅 GCP 云服务	开源 / 自建 / 云	开源 / 自建 / 云
成本	较高	开源免费	开源免费
适用场景	全球金融 / 电商	MySQL 迁移 / HTAP	PostgreSQL 迁移

59 HTAP：混合事务与分析处理

西南交通大学 Southwest Jiaotong University

HTAP 在同一系统中同时支持事务处理（**OLTP**）与分析处理（**OLAP**），无需将数据经**ETL**同步到单独的数据仓库。

OLTP=联机事务处理（高并发增删改查） **OLAP**=联机分析处理（大范围聚合统计） **ETL**=抽取-转换-加载（将业务数据批量搬运到数仓的流程）

■ 传统架构 vs HTAP 架构对比

❌ 传统方式（分离架构）

OLTP 数据库
MySQL / PostgreSQL
→ **ETL 批量同步**
T+1 延迟（小时~天）
→ **OLAP 数据仓库**
Hive / ClickHouse

⚠️ 数据延迟 T+1，ETL 维护复杂，存储成本翻倍

✅ HTAP 方式（TiDB）

TiKV 行存
OLTP 写入
实时事务
⇒ **TiFlash 列存**
OLAP 分析
异步复制

Raft Learner 实时同步

✅ 实时一致，无 ETL，一套 SQL 查询全部数据

■ 典型 HTAP 应用场景

实时风控
交易发生时即时计算风险分数，无需等待数据同步到数仓

实时报表
业务数据写入同时，报表查询立即可见，T+0 而非 T+1

混合负载
OLTP 写入高峰期，TiFlash 承接分析查询，互不干扰

分布式存储全景图

类别	代表系统	数据模型	一致性	典型场景
KV 存储	Redis, DynamoDB	Key-Value	AP/CP	缓存、会话、配置
文档存储	MongoDB	JSON 文档	可调	内容管理、用户画像
列族存储	HBase, Cassandra	Column Family	CP/AP	时序、日志、大数据
图数据库	Neo4j, JanusGraph	图（节点 + 边）	CP	社交网络、知识图谱
NewSQL	TiDB, Spanner	关系表	CP	金融、电商核心业务
时序数据库	InfluxDB, TimescaleDB	时间序列	AP	监控、IoT

67 课堂互动与思考题

西南交通大学 Southwest Jiaotong University

CAP选择：你正在设计一个全球电商系统的购物车服务，用户可能从不同地区访问。你会选择CP还是AP系统？为什么？如果是支付服务呢？

一致性哈希：一致性哈希环上有4个节点，每个节点100个虚拟节点。如果删除一个节点，大约有多少比例的数据需要迁移？迁移到哪里？

存储选型：你的系统需要存储10亿条用户行为日志（每天新增1000万条），需要支持按用户ID查询最近30天的行为记录。选择什么存储系统？为什么？

NewSQL vs NoSQL：为什么不直接在所有场景使用TiDB/Spanner这样的NewSQL数据库，而还需要Redis、Cassandra等NoSQL？

68 课堂互动与思考题

西南交通大学 Southwest Jiaotong University

用Python实现一致性哈希，测试不同虚拟节点数量下的数据均匀度

使用Docker部署Redis Cluster（3主3从），验证数据自动分片

(选做)部署TiDB Playground，测试MySQL兼容性

69 本讲知识要点回顾

西南交通大学 Southwest Jiaotong University

模块	核心知识点
CAP 定理	C/A/P 三选二（实际是 P 下选 C 或 A）；PACELC 扩展
BASE 理论	基本可用、软状态、最终一致 ——AP 系统的设计理念
一致性模型	线性一致→顺序一致→因果一致→最终一致
数据分片	哈希分片 vs 范围分片；一致性哈希 + 虚拟节点
数据复制	主从 / 多主 / 无主复制；Quorum 机制（W+R>N）
KV 存储	Redis Cluster（哈希槽）；DynamoDB（全托管）；etcd（Raft 强一致）
列族存储	BigTable 模型；HBase（CP）；Cassandra（AP，可调一致性）
NewSQL	Spanner（TrueTime）；TiDB（MySQL 兼容）；HTAP

70 下次课预告 & Q&A

西南交通大学 Southwest Jiaotong University

下次课：第8讲 分布式文件系统

- GFS / HDFS架构与原理
- 块存储 vs 对象存储 vs 文件存储
- 对象存储（MinIO / S3）
- Ceph统一存储
- 分布式文件系统对比

预习建议：

- 了解HDFS的基本概念（NameNode、DataNode）
- 了解对象存储（S3）的基本使用

71 课后作业

西南交通大学 Southwest Jiaotong University

用Python实现一致性哈希环，测试增删节点时的数据迁移比例

比较Redis Cluster、DynamoDB和Cassandra在数据分片策略上的差异

分析你常用的互联网应用（如微信/淘宝），推测其后端可能使用了哪些分布式存储技术